DishaAI: AI-Powered Accessible City Signage

PROJECT WRITE-UP | Team ByteForge | Megathon IIIT Hyderabad 2026

1. Problem Statement and Understanding

Cities contain important information on traffic signs, bus boards, shop boards, advertisements and direction signs. People need to understand this information to move around and make everyday decisions. However, the information is often presented visually or in text that may be difficult to read, understand or access. This creates barriers for people with visual impairments and can also affect people who have hearing or hand-motor impairments when information and controls are not designed in an accessible way. Language differences and unclear photographs can make the problem harder.

Our understanding is that making a sign digitally readable is only one part of the solution. The information must also be communicated in a format the user can access: spoken output for users who cannot easily read a sign, visible text and captions for users who cannot rely on audio, and simple controls for users who have difficulty with precise hand movements. The system should be clear about uncertain results and ask the user to try again when a sign cannot be read reliably. It should not invent missing words or present uncertain sign recognition as fact.

2. Idea / Abstract

DishaAI is a proposed web application that helps users understand common city signs using image-based text recognition, simplified explanations, multilingual support and accessible speech features. A user can capture or upload a photograph of a signboard. Optical Character Recognition (OCR) extracts the visible text, after which an AI component is intended to explain the text in short, simple language. The application is planned to support English, Telugu and Hindi, with speech recognition, translation and text-to-speech integrated through the official BHASHINI services.

The prototype focuses on three accessibility needs. For visual impairment, it prioritizes spoken explanations and concise cues. For hearing impairment, it provides readable text, captions and visual status messages. For hand-motor impairment, it uses large buttons, a simple interface and voice controls where supported. If a photo is blurry, the text is unclear, or a service request fails, the application will display a helpful message and allow the user to retry. The aim is to make everyday city information easier to access, while keeping the first version focused on reliable sign understanding rather than promising a complete navigation system.

3. High-Level Approach, Pipeline and Architecture

DishaAI will use a web frontend, a Python backend and external AI/language services. HTML, CSS and JavaScript will provide the interface, including image upload, language selection, accessible result display and speech controls. Python with FastAPI will receive uploaded images and coordinate the processing steps. Tesseract OCR will be used initially to extract text from the image. An AI service will generate a simple explanation based on the extracted text, while BHASHINI services will be used for the required speech recognition, translation and speech synthesis pipeline. API credentials will be stored on the backend in environment variables, not in frontend code or the public repository.

Planned processing pipeline

- **1. Capture:** The user uploads or captures a photo of a traffic sign, bus board, shop board or direction sign.
- **2. Validate:** The backend checks the file type and size and returns a clear error if the image cannot be processed.
- **3. Recognize:** OCR extracts visible words from the sign. If the result is empty or too unclear, the user is asked to retake the photo.
- **4. Explain:** An AI component produces a concise, plain-language explanation grounded in the recognized text.
- **5. Translate and speak:** The selected language is used for translation and speech output through the official BHASHINI pipeline, subject to service availability and valid credentials.
- **6. Present accessibly:** Results are shown as readable text and, where available, captions, spoken output and adjustable text/speech settings.
- **7. Recover:** If OCR, speech recognition, an API call or the network fails, the app shows a helpful message and provides a retry path.

At a high level, the architecture is: **Browser interface** → **FastAPI backend** → **OCR** → **AI explanation** → **BHASHINI language/speech services** → **accessible text and audio output.** The prototype will be built in stages so that image upload and OCR can be tested before the AI and speech integrations are added. Any feature that depends on event credentials will be reported as pending until it has been connected and tested successfully.

4. Planned Testing Methodology

Testing will be performed incrementally using a small set of real or prepared signboard photographs, including traffic signs, bus boards, shop signs and direction signs. The same examples will be used across repeated trials to identify errors and compare outputs. Testing will focus on functional correctness, accessibility, language support and failure recovery rather than claiming performance that has not been measured.

- **OCR tests:** Try clear, distant, tilted, low-light and blurry images. Record whether the important text is extracted correctly and whether unclear cases trigger a retake message.
- **Explanation tests:** Check that explanations are short, understandable and consistent with the extracted text, without adding unsupported directions or details.
- **Language and speech tests:** Test English, Telugu and Hindi selection. Verify the BHASHINI speech-recognition, translation and text-to-speech stages with valid credentials and record failures or incorrect outputs.
- **Accessibility tests:** Check keyboard navigation, screen-reader labels, contrast, text resizing, visible captions, large controls and adjustable speech speed. Ask volunteers to try the relevant flows when feasible.
- **Reliability tests:** Test unsupported files, oversized images, empty OCR results, API errors and interrupted network connections. Confirm that the app responds clearly and does not freeze or display fabricated results.
- **Evaluation record:** Maintain a test table with test case, expected result, observed result, pass/fail status and notes. Fix high-priority issues and rerun failed cases before the demonstration.

Expected outcome: A demonstrable web prototype that recognizes signboard text, explains it simply and presents information through accessible text and speech pathways. Final claims will be based on the features and tests completed by the team.